

4C11 Coursework 1

5630C

March 19, 2025

Abstract

Two physical problems demanding solutions to second-order differential equations are solved using Physics-Informed Neural Networks (PINNs). In problem 1, a loss function based on data examples is shown to be superior to loss based on PDE and boundary conditions with no data. In problem 2, a Fourier Neural Operator network is shown to be superior to a CNN (U-Net) in solving a Darcy flow problem. Network design, hyperparameter tuning and performance is discussed.

Problem 1

Method

Design A solution to a single loading case of a bi-symmetric plate is desired. One corner section of the plate is taken as the system, as shown in the coursework handout. The equations describing the system of the flat plate section are given in eq. (1). h is the boundary condition on normal stress at edges (boundary shear of zero is also enforced) and g is the boundary condition on displacement at edges.

$$\begin{aligned}\varepsilon &= \frac{1}{2}(\nabla u + \nabla u^T) && \text{Strain tensor} \\ \sigma &= C\varepsilon && \text{Stiffness matrix} \\ \sigma \cdot n &= h && x \in \partial\Omega \\ u &= g && x \in \partial\Omega \\ \nabla \cdot \sigma &= 0 && x \in \Omega\end{aligned}\quad (1)$$

PINNs are known to have difficulty with second-order derivatives in the loss function because they introduce sharp variations that amplify errors. To avoid double differentiation, two neural networks are used: one to compute predicted displacement $\hat{u}$ and one to compute predicted stress $\hat{\sigma}$, as given in eq. (2).

The overall loss L is made up of:

- Boundary loss L_b , computed using $\hat{u}$ and $\hat{\sigma}$
- Divergence loss L_{div} , computed using $\hat{\sigma}$
- A term to ensure consistency between the two networks L_{map}

MSE is the Mean-Square Error loss function.

$$\begin{aligned}\text{NN}_1 &=: x \mapsto \hat{u}(x) \\ \text{NN}_2 &=: x \mapsto \hat{\sigma}(x) \\ L_b &= \text{MSE}(\hat{\sigma} \cdot n, h) + \text{MSE}(\hat{u}, g) \\ L_{div} &= \text{MSE}(\nabla \cdot \hat{\sigma}, 0) \\ L_{map} &= \text{MSE}\left(\hat{\sigma}, C \frac{1}{2}(\nabla \hat{u} + \nabla \hat{u}^T)\right) \\ L &= L_b + L_{div} + L_{map}\end{aligned}\quad (2)$$

Training Two training cases were tested to compare between performance of pure physics-based loss function and data-based training. First, the network was trained on the pure physical loss function as described above. Second, the network was trained on the same loss function appended with a term based on 50 ground truth displacement datapoints. This term was scaled by 100x to make it dominant in training.

The optimiser used was LBFGS, which has been shown to consistently outperform the Adam optimiser in PINN tasks [1]. The number of loss evaluations per epoch for both tests was 10, and the number of epochs was 500, meaning 5 thousand total training steps. This is much smaller than the advised 50-100 thousand and was limited by the speed of my computer; longer training durations were unreasonable (and unnecessary for this simple comparison).

The only hyperparameter tuned was the LBFGS learning rate.

Results and Discussion

Note, "PDE loss" in this section refers to the total loss without data, L . "Data loss" refers to the loss with the appended displacement error term.

The loss curves for the two training cases are shown in fig. 2 and the resulting predicted stress distributions are shown in fig. 3. The ground truth stress is shown in fig. 1. The optimal LBFGS learning rate was 1.

To the eye, training with the loss function based mainly on data rather than the PDE seems to massively improve the accuracy of the results (fig. 3). Notably, the stress concentration at the top of the cutout becomes very close to the ground truth value where it was poorly predicted before data. This is reflected in the data loss, which is 3 orders of magnitude smaller for the data-based training. However, the PDE loss for the data-based training seems worse. This demonstrates that PDE loss may not be a good proxy for prediction accuracy of the desired variable, here u , highlighting a major difficulty with training

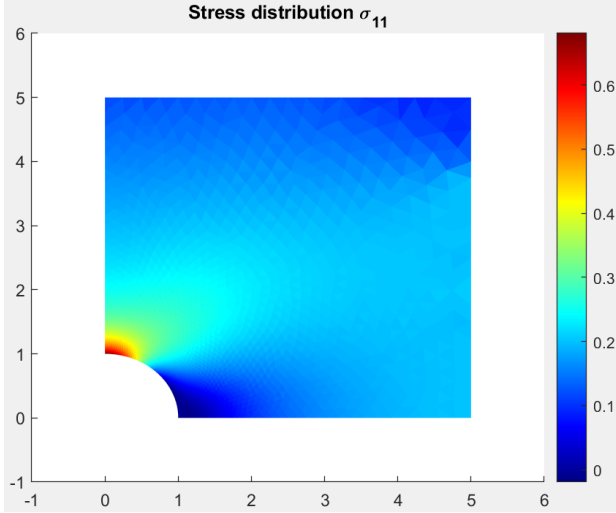


Figure 1: The true stress distribution, generated using the course-work materials. A key metric is the stress concentration at (0,1).

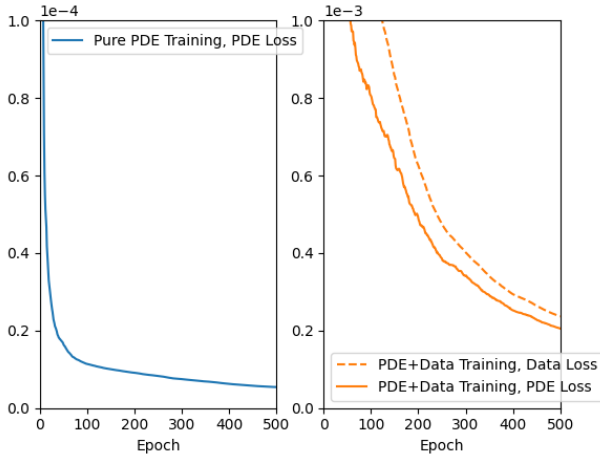


Figure 2: The PDE-based training loss for the two training cases. For the data-based training case, the data-based loss is also shown. The results would have benefited from more epochs but this was limited by computation time.

PINNs on pure PDEs.

The problem scenario, with only a single load case, is somewhat contrived. Training a PINN for a single load case presumably means there is no data to train with, so the pure PDE loss function approach would be necessary which has been shown to perform poorly and takes longer to train than computing a finite element solution regardless. The PINN is unlikely to generalise to other load cases without further training each time. The major insight from this investigation is the superiority of training on data to training on the PDE with boundary conditions.

Problem 2

A Darcy flow problem, a 2nd order differential equation with input and output of shape 32x32, is used

to compare the performance two PINN architectures: a CNN-based network (U-Net) and a Fourier Neural Operator (FNO).

U-Nets, based on CNNs but with additional skip-connections between layers of the same scale, are commonplace in applications with similar input and output forms (such as image transformation), so are expected to work well for this problem.

FNOs are designed to solve differential equations. Instead of learning CNN kernels, FNOs parametrise and learn continuous kernel functions that act on a high-dimensional "lifted" transformation of the input.

A major advantage of FNOs is to provide some degree of scale invariance, that is providing accurate predictions for different input and output resolutions, including ones not trained on previously. This capability is not assessed in this investigation (all the inputs and outputs are shape 32x32). Instead, this investigation will demonstrate whether FNO has superior performance to U-Net disregarding this advantage.

Method

The U-Net architecture used is shown in fig. 4. The FNO architecture is the same as that in the skeleton code. The U-Net had 1.9 million parameters; the FNO had between 1-2 million depending on the hyperparameters. The AdamW optimiser with zero weight decay was used to train both.

The learning rate was tuned for both networks. For the FNO, the number of Fourier modes and dimension of the lifted space was also tuned. The FNO was also tested with and without a scheduler.

Results and Discussion

The loss plots for the U-Net and variants of the FNO are shown in fig. 5. Two predictions on examples from the test set, compared with the ground truth, are shown in fig. 6.

The U-Net could clearly have benefitted from more training epochs, though the training loss seems to start to plateau around 90 epochs. The FNO clearly has superior performance to the U-Net, as shown by the loss and the visual plots, despite the "vanilla" (fig. 5) architecture having half as many parameters (1 million) as the U-Net. The hyperparameter variants of the FNO all seemed to plateau at a mean L2 loss of around 0.045.

The most effective learning rate was found to be 0.0001 for the U-Net. For the FNO, 0.001 was optimal without the scheduler and 0.0005 with.

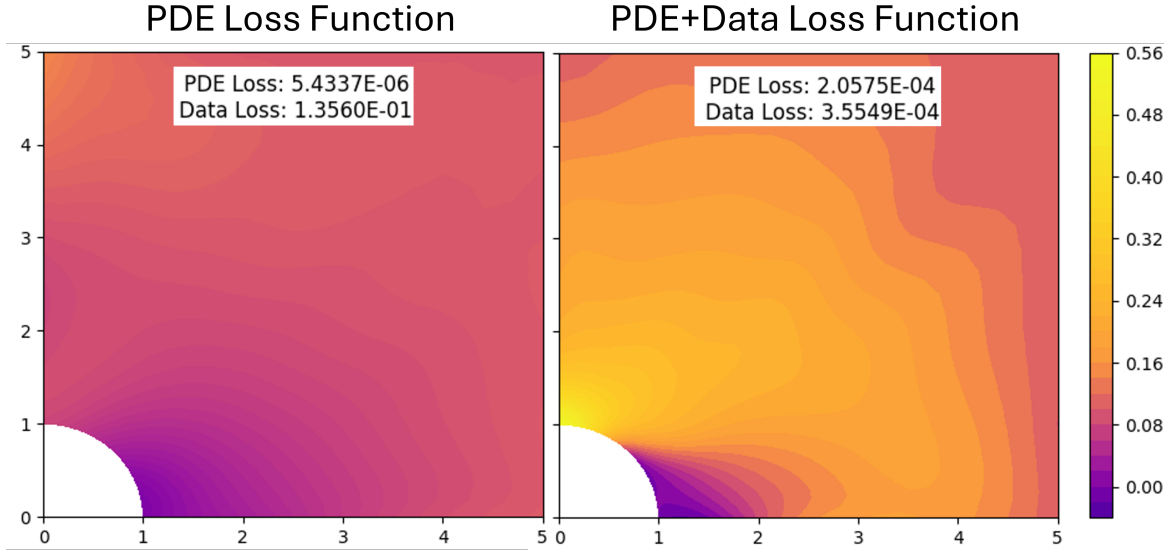


Figure 3: σ_{11} predictions to be compared against the ground truth in fig. 1. The stress concentration at (0,1) was predicted to a more accurate extent by the data-based training test.

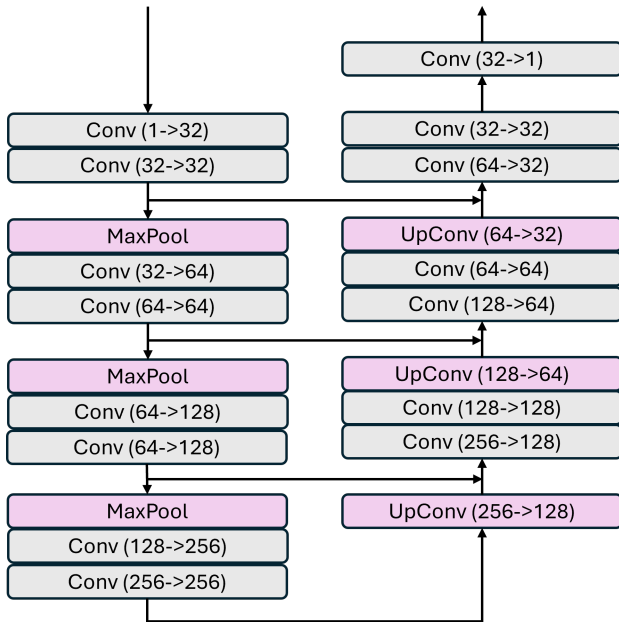


Figure 4: The U-Net architecture. Horizontal arrows indicate skip-connections, where channels are concatenated. ReLU activation was used after every convolution layer (other than the output convolution), including up-convolution. The arrows indicate changes in channel size. Max-pooling and up-convolution layers all have kernel size 2x2 and step size 2.

Hyperparameter Tuning Strategy

In the previous coursework, it was demonstrated that batch size is unlikely to affect the true "potential" of a network (the minimum loss at the training plateau). The effect of increasing batch size is to increase per-epoch training speed and stability while requiring more epochs to reach a certain loss. Batch size was chosen to minimise training time, considering this tradeoff.

AdamW was always favoured as the optimiser, due to its high performance in a wide range of tasks and the lack of need for a scheduler. It is easy to tune the learning rate (it usually falls within 0.001 - 0.00005) and the other hyperparameters of AdamW rarely benefit from tweaking. A departure from this was the use of LBFGS in Problem 1. This was due to strong evidence from the literature of its superiority in PDE-loss PINN tasks and far superior qualitative performance on Problem 1 in early tests (which are not documented in this report).

None of the loss curves demonstrated overfitting (a disconnect between training loss and validation loss), so regularisation was not considered.

The above considerations mean learning rate of the given optimiser was often the only tuned hyperparameter. Learning rates were always chosen as a trade-off between search power (ability to decrease the loss quickly) and training stability. Learning rates were increased until the training loss decrease became slower or altogether unstable (characterised by frequent large spikes in loss, and sometimes occasional erasing of progress).

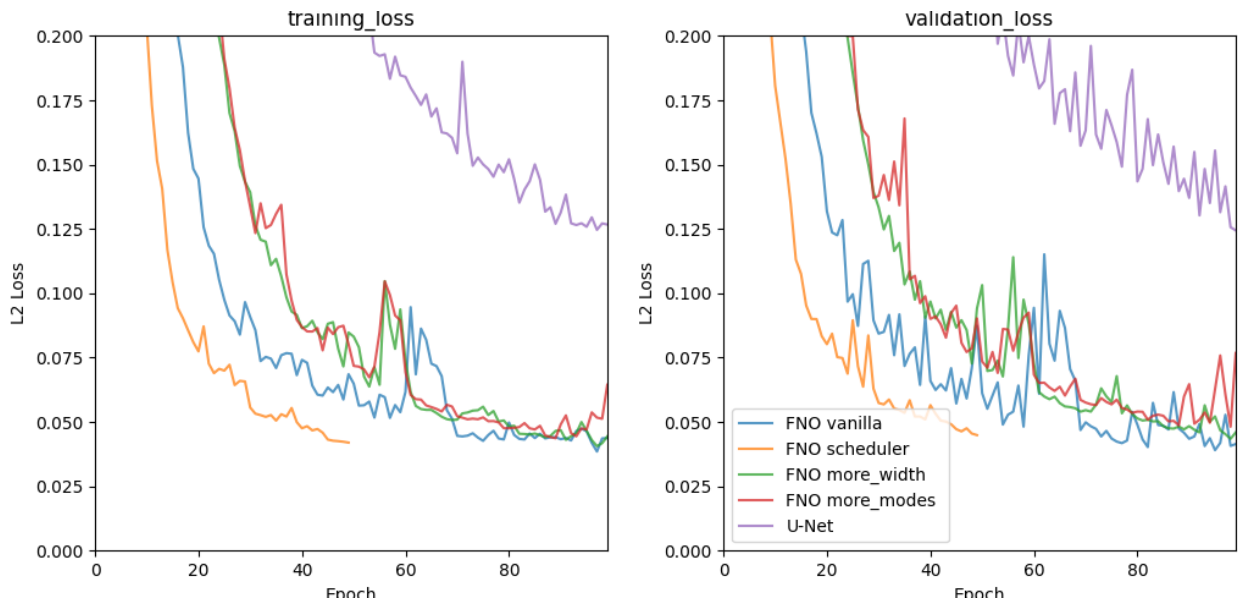


Figure 5: Loss plots for the networks. "Vanilla" means the FNO has 12 fourier modes and lifted width 32 (the default in the skeleton code). Using the provided scheduler as well as AdamW (the other runs used no scheduler) produced "scheduler". The following two tests also used the scheduler: increasing the fourier modes to 16 produced "more modes"; increasing the width to 40 (12 fourier modes) produced "more width".

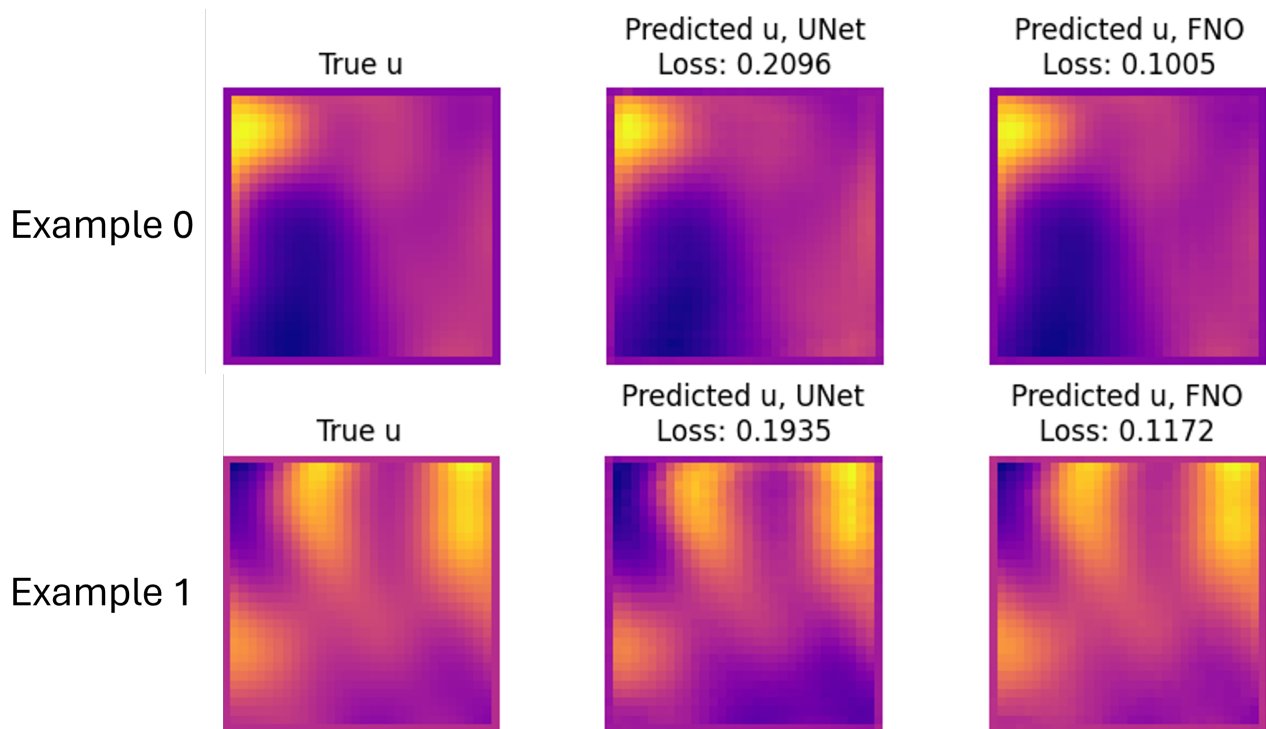


Figure 6: *cap*

References

- [1] Pratik Rathore et al. “Challenges in training PINNs: A loss landscape perspective”. In: *arXiv preprint arXiv:2402.01868* (2024).